Open Access | https://doi.org/10.25080/ymuf5798



SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Reno: Simplifying Application of Bayesian Inference to System Dynamics

Nathan Martindale[1], **Jordan Stomps**[1], and **Birdy Phathanapirom**[1]

[1]Oak Ridge National Laboratory

Abstract

Modeling and simulation enable iterative testing and updating of hypotheses, while encoding subject matter expertise into reusable tools. While the Python community has a variety of libraries for modeling, few exist for system dynamics, a paradigm for top-down analysis of material and information flows over time. Reno is an open-source package combining creation, visualization, and analysis of system dynamics models with techniques for Bayesian inference through integration with PyMC, supporting probability distributions in system variables and MCMC sampling to produce posterior distributions based on observed values. This approach enables simulation and refinement of time series models where variables, policies, or knowledge are uncertain, and data/observations are sparse.

Keywords System dynamics, Bayesian statistics

1. Introduction

Throughout the history of computer science, computers have been used to model aspects of the world in a diverse range of fields, from physics [1] and biology [2] to economics [3] and sociology [4]. Studying a phenomenon via computation-based tools allows exploration of hypotheses in a simplified environment, which is often necessary when real-world data are expensive or difficult to obtain. Computational simulations can be used as a starting point for identifying opportunities for further research (e.g., what real-world data might be worth collecting), helping to inform policy decisions, or even standing independently as valuable research contributions. Furthermore, computational models represent a mathematical interpretation or understanding of the world, encoding knowledge, ideas, and prior research from subject matter experts.

Although many techniques for conducting modeling and simulation exist, this work focuses on two specific approaches and how they can be combined: system dynamics and Bayesian statistics. To this end, we present Reno[1], an open-source Python library that implements an API for creating system dynamics models that can be wrapped in a Bayesian statistical model with PyMC [5] for updating uncertain parameters based on observed data.

1.1. System Dynamics Modeling

System dynamics modeling (SDM) is a way to represent how material and information move through and accumulate within complex systems. These systems are characterized by stocks (states where material or information accumulates) and flows (equations that define how those accumulations increase or decrease over time) and often exhibit nonlinear behavior arising from feedback loops and time delays [6]. SDM explores top-down impacts to system behavior, as opposed to a paradigm like agent-based modeling which explores emergent phenomena in a system from a bottom-up encoding of small-scale interactions

[1]https://github.com/ornl/reno

Published Jul 13, 2026

Correspondence to
Nathan Martindale
martindalena@ornl.gov

Open Access

Copyright © 2026 Martindale *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

[7]. SDM was first developed by Jay Forrester in 1956 to understand how corporate structure impacted employment oscillations at a General Electric plant [6], [8]. The approach has since been applied in many different domains, particularly within supply chain management and industrial production [9], [10], [11] as well as fields such as natural resource management [12], [13] and interactions between social and ecological systems [7].

For the purposes of a simple example to illustrate how stocks and flows interact in a system, consider a bathtub. The tub itself represents a stock, measuring the amount of water that has accumulated over time. The faucet and drain would be represented as an inflow and outflow to the tub respectively, influencing the rates at which the water level changes (Figure 1). Nonlinearities are introduced with concepts like feedback loops, such as a water level sensor that connects to the faucet and automatically increases or decreases the flow rate to maintain a certain water level in a leaky tub. Time delays are another common source of complexity, where the transmission of values from specific variables/flows may lag many timesteps behind. To overextend the tub analogy, one could imagine if the water level sensor is a poorly designed, wifi-enabled, Internet of Things (IoT) device with a sensor reading that is always several minutes behind, then it leads to an oscillating faucet rate as it cyclically undershoots and overshoots the target level.

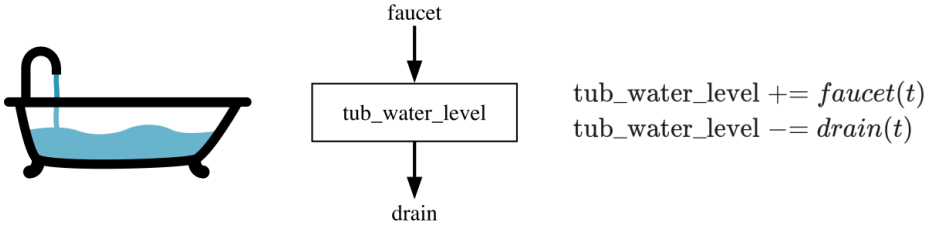


Figure 1. *Basic tub system. The* `tub_water_level` *rectangle represents the stock, and the* `faucet` *and* `drain` *arrows represent inflow/outflow. The stock value at each simulated timestep is increased/decreased by the rates of the inflows/outflows at time* `t`*.*

A more traditional example, the Lotka-Volterra equations or "predator-prey" model, shows two interacting populations of animals, a prey and predator group. The model exhibits several feedback loops, for instance the exponential growth between a population and the population's birth rate (i.e., as the population grows, birth rate grows, causing the population to grow even faster). Interaction between the two populations occurs in the predation loop, shown in the middle of Figure 2, where prey sustains the predator population but decreases the prey population, thus preventing unbounded growth from either population's positive feedback loop. Causal loop diagrams like the one shown only highlight general interactions in a system, but the specific values used in variables associated with these concepts impact the overall behavior.

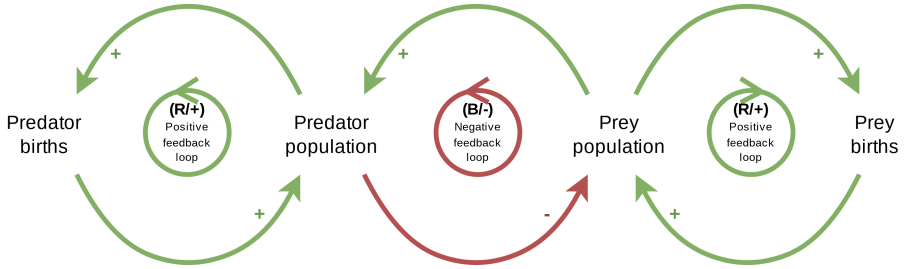


Figure 2. *An example causal loop diagram (commonly used to highlight feedback loops) of interactions between two populations. "R" (reinforcing), or positive feedback loops increase exponentially; "B" (balancing), or negative feedback loops, oscillate or push towards homeostasis.*

1.2. Bayesian Statistical Modeling

Bayesian inference is an approach to statistical modeling in which unknown quantities such as model parameters are represented with probability distributions. These distributions are then updated and refined based on observed data. This approach promotes an intuitive way of considering uncertainty and can produce meaningful information even with sparse input data. The Bayesian statistics interpretation of uncertainty is often contrasted with the frequentist interpretation: a useful distinction drawn from [14] is that the Bayesian approach considers the uncertainty of hypotheses (the model and its parameters) rather than the uncertainty of the underlying data like in the frequentist approach. This aligns well with SDM, in which different structures and parameterizations function as different hypotheses. For models with two possible pathways or structures (achievable by switching flows between different subsections of a model through the use of piecewise equations based on a bernoulli distribution), applying Bayesian inference can update probability distributions for whichever underlying model structure is more likely based on a set of observed data.

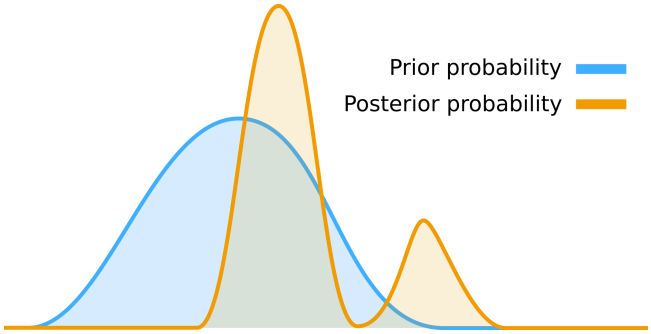


Figure 3. *Example probability distributions before and after Bayesian inference. These probability distributions could represent what the likely value of a particular parameter is. The prior shows a wider, "uncertain" value, whereas the posterior (the updated distribution after trying to fit some data) shows tighter probability mass around two specific likely values.*

This update process is theoretically underpinned by Bayes' theorem, shown in Equation (1), a formula that computes a posterior probability (the probability of a hypothesis conditioned on some data) based on a combination of the prior probability and the probability of observing that specific data. In practice, using Bayes' theorem directly is often computationally infeasible, so sampling-based approximations are typically used instead [14]. These approximations come from Markov Chain Monte Carlo (MCMC) algorithms [15], or methods of sampling from a probability distribution. MCMC sampling techniques were first developed in the 1940's, but began to see use in Bayesian approaches starting in the 1990's [16].

$$P(\text{hypothesis}|\ \text{data}) = \frac{P(\text{data}|\ \text{hypothesis})P(\text{hypothesis})}{P(\text{data})} \tag{1}$$

1.3. Combining SDM and Bayes

When constructing a system dynamics model, some parameter values may be unknown or uncertain. A phenomenon could be modeled by several different SDM structures or pathways. Because each has ranges of possible values for input parameters, the complexity of the phenomenon may mean there is no obvious appropriate structure for closely modeling real-world data. Therefore, applying Bayesian statistics in these situations allows the use of probability distributions which represent prior knowledge and uncertainty of unknown parameters. If provided some piece of real-world data about an observable part of the system, Bayesian inference can approximate the posterior distributions of those parameters and subsequently calibrate the model to the observed data.

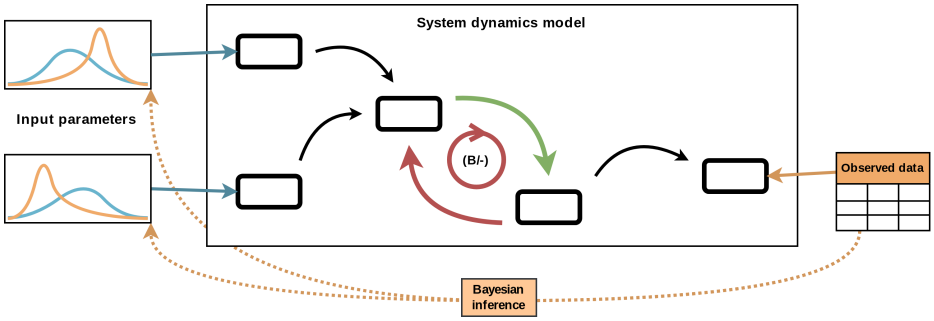


Figure 4. *Input parameters to a system influence some other component within the system. Providing observed data points for a piece within the system allows the use of Bayesian inference to learn/reduce uncertainty in the input parameters needed to produce the observed data.*

Reno seeks to implement an efficient workflow that combines these two modeling paradigms. The combination of SDM with Bayesian inference in and of itself is not novel: the process of estimating parameter distributions based on data is supported in the proprietary Vensim software, and has been used in literature for calibrating with sparse data [17] as well as forecasting with uncertainty from a base model calibrated on real data [18]. As such, Reno's contribution is in supporting within the Python open-source ecosystem the full process of developing, visualizing, and debugging system dynamics models, as well as simulation that incorporates Bayesian inference.

2. Existing tools

A variety of tools and libraries exist for system dynamics, many of which are proprietary. Some well-known solutions in industry and literature include AnyLogic[2], Vensim[3], and Powersim[4]. Other software ecosystems include open-source libraries and tools, such as the web-based/JavaScript community. Insight Maker [19] is an excellent example that includes both a back-end API to create models and graphical front-end that intuitively supports creating and connecting the various SDM components and defining subsequent equations. Along with SDM, Insight Maker supports agent-based modeling, although it has no direct mechanisms for conducting Bayesian inference. StatSim[5], another JavaScript front-end tool, also has direct support for both SDM and Bayesian inference applied to the system dynamics models.

Tools for SDM within the scientific Python community are currently limited: at the time of writing, there are only two notable libraries that support it. PySD [20], the more established of the two, is designed to run and modify SD models that have been written in other tools such as Vensim. However, it cannot build a model from scratch. PySD supports Bayesian inference through a manual integration with PyMC, whereby it treats the underlying system dynamics model as a black box function rather than translating the model into a PyMC equivalent. BPTK-Py (business prototyping toolkit)[6] is an open-source framework for creating SDM and ABM models, but does not support Bayesian inference. A comparison of these tools is listed in Table 1.

Table 1. *Comparison of a small selection of SDM tools. "Usage" refers to whether its main functionality is through a graphical user interface or a particular programming language API.*

Tool	Open-source	Usage	Create SD models	Bayesian inference

[2] https://www.anylogic.com/
[3] https://vensim.com/
[4] https://powersim.com/
[5] https://statsim.com/
[6] https://github.com/transentis/bptk_py

Vensim	No	Graphical	Yes	Yes
Insight maker	Yes	JavaScript/Graphical	Yes	No
StatSim	Yes	JavaScript/Graphical	Yes	Yes
PySD	Yes	Python	No	Yes
BPTK-Py	Yes	Python	Yes	No
Reno	Yes	Python	Yes	Yes

Reno was developed in part because implementation of these capabilities directly in Python allows for more effective integration into other libraries, tools, and frontends, such as Jupyter Notebooks, custom CLIs, or experiment management software. A Python API further lends itself to easier comprehension and modification via LLMs.

3. Using Reno

There are four basic aspects to the Reno API. These include model objects, the system dynamics components added to a model, the symbolic math system that is used to construct the equations of the components, and the collection of utilities for converting to PyMC models, visualization, and graphical explorer.

3.1. Creating a model

A model is constructed by initializing a `Model` instance and adding components to it, shown in Program 1. Components (e.g., stocks, flows, configurable variables, etc.) are added either by assigning directly to attributes on the model object, or by creating components within the model's context manager similarly to in PyMC.

```
import reno as r

my_model = r.Model()

# one way to add components is to directly assign them as attributes
my_model.variable1 = r.Variable()

# the other way (to avoid typing the model name over and over)
# is to use the model as a context manager. Any components defined
# inside of the context manager get added to the model when the
# context manager exits
with my_model:
  variable2 = r.Variable()
  flow1, flow2 = r.Flow(), r.Flow()
  my_stock = r.Stock()

my_model.my_stock  # components defined within the context manager are
           # available as attributes on the model
```

Program 1. *Basic example of creating a model and adding a couple variables (placeholder values that can be used in other equations), flows, and a stock.*

Equations are added to components by defining them within the constructor of the component, or by setting the `.eq` attribute, as shown in Program 2. The ability to set an equation outside of the component constructor allows certain kinds of circular references necessary for implementing feedback loops. Evaluating an equation that includes a reference to another component will subsequently evaluate the other component's equation (Figure 5). Circular references are allowed specifically for stock references, where using a stock's value always refers to the previous timestep's value which was already computed.

```python
import reno as r

tub = r.Model("Tub model")
with tub:
  faucet = r.Flow()
  drain = r.Flow()
  water_level = r.Stock()

  # equations can be specified in the constructor
  final_water_level = r.Metric(water_level.timeseries[-1])

  # or they can be defined after instantiation
  drain.eq = water_level / 2
  faucet.eq = 6

  # stocks are defined in terms of adding/subtracting flows
  # to make them inflows/outflows
  water_level += faucet
  water_level -= drain
```

Program 2. *Example bathtub model showing the two ways equations can be defined for different components. Stock equations are defined indirectly by adding/subtracting flows.*

An equation in Reno is fundamentally a tree data structure, where any node that has subtrees is an operation and leaf nodes are references to other equations (other stocks/flows/variables in the model) or values. An equation is evaluated by recursively evaluating through the tree down to the leaf nodes, and in turn returning produced values back up through the tree. Each operation (a set of classes included with Reno) has definitions for their evaluation based on its subtrees both in numpy [21] as well as how to construct the equivalent subtree in PyMC (Figure 5). Similar to PyTensor and numpy, all basic math operators are overloaded to simplify the resulting Python model code and contain the math to within Reno's symbolic math approach.

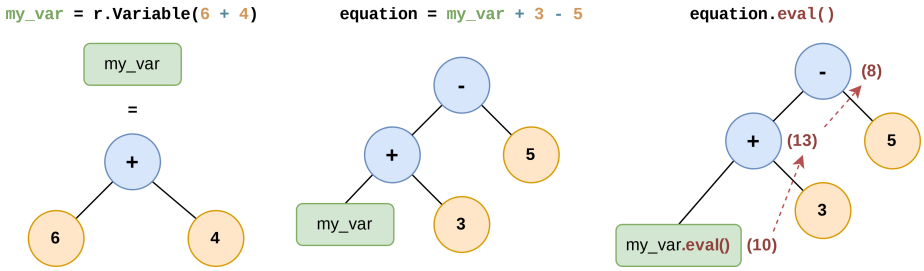


Figure 5. *An example of how an equation tree is structured and evaluated. Evaluation recurses through the tree, including other trees if there are references to other components, to the leaf nodes. Operation results propagate back up to the root.*

3.2. Running a model

A model, once defined, can be called as a function to run a simulation. This function call can include simulation-specific configuration parameters, such as the number of steps to run, and can also be used to configure any free variables in the system (i.e., variables not defined based on other variables). This allows quickly building up a collection of datasets from multiple different configurations. Results are returned in XArray datasets that contain the values of every component within the system indexed by timestep and sample number. These datasets can be passed into many of Reno's available visualization functions.

The following example in Program 3 implements the Lotka-Volterra equations and executes two different simulation runs with them. Each run is configured slightly differently:

```
import reno as r
predator_prey = r.Model(name="predator_prey", steps=200, doc="Classic predator-prey interaction model
example")

with predator_prey:
    # make stocks to monitor the predator/prey populations over time
    rabbits = r.Stock(init=100.0)
    foxes = r.Stock(init=100.0)

    # free variables that can quickly be changed to influence equilibrium
    rabbit_growth_rate = r.Variable(.1, doc="Alpha")
    rabbit_death_rate = r.Variable(.001, doc="Beta")
    fox_death_rate = r.Variable(.1, doc="Gamma")
    fox_growth_rate = r.Variable(.001, doc="Delta")

    # flows that define how much the stocks change in a timestep
    rabbit_births = r.Flow(rabbit_growth_rate * rabbits)
    rabbit_deaths = r.Flow(rabbit_death_rate * rabbits * foxes, max=rabbits)
    fox_deaths = r.Flow(fox_death_rate * foxes, max=foxes)
    fox_births = r.Flow(fox_growth_rate * rabbits * foxes)

    # hook up inflows/outflows for stocks
    # note that `>>` is syntax sugar for the appropriate `+=`/`-=` stock operations
    rabbit_births >> rabbits >> rabbit_deaths
    fox_births >> foxes >> fox_deaths


run1 = predator_prey(rabbit_growth_rate=.06, rabbits_0=50.0)
run2 = predator_prey(rabbit_growth_rate=.04, rabbits_0=75.0)
```

Program 3. *Reno implementation of the predator-prey equations (labeled as foxes/rabbits.) Model parameter configuration can include initial values of stocks by suffixing with `_0`, such as `rabbits_0`.*

3.3. Visualizing a model

Reno includes several functions to help visualize and debug models. One function includes a stock and flow diagram, which visually lays out how the components connect to each other and highlights the main material/information flow(s) throughout the system (Figure 6).

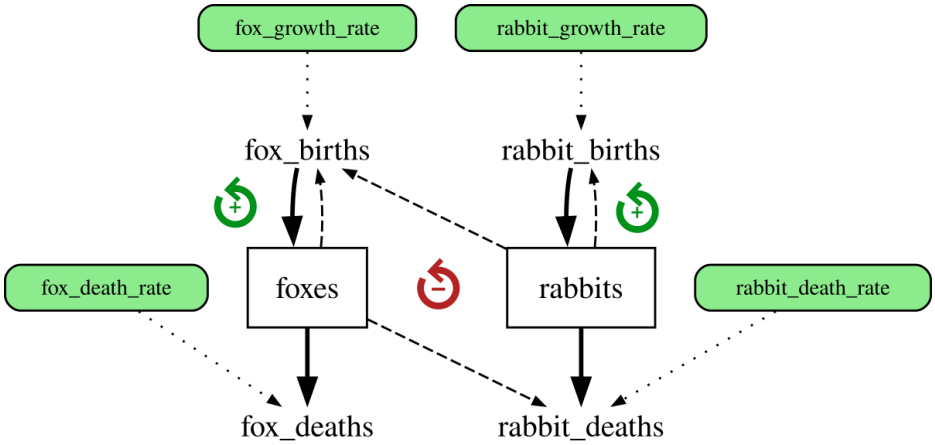


Figure 6. *The output from `predator_prey.graph()`, a graphviz stock and flow diagram. Rectangles represent stocks, labeled arrows are flows, and green rounded rectangles are variables. Feedback loop indicators were added manually to show how a stock and flow diagram relates to the causal loop diagram in Figure 2.*

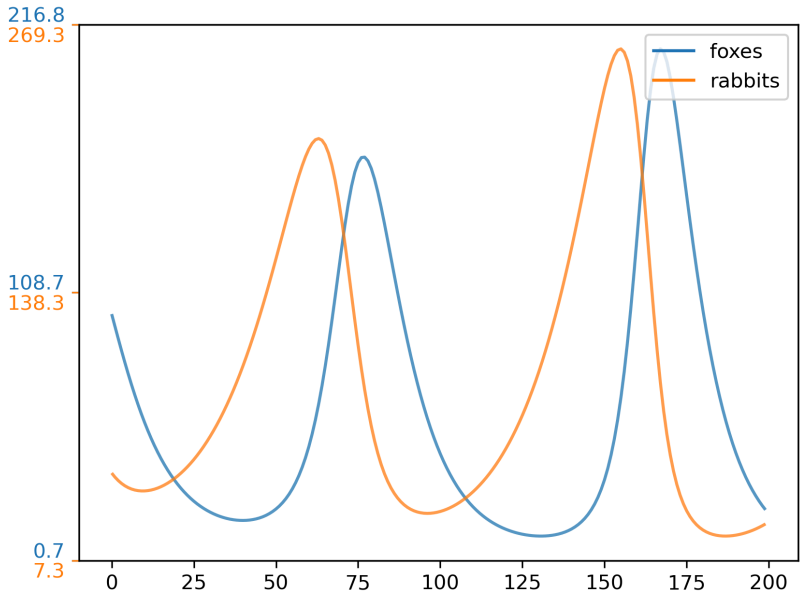


Figure 7. *The output from* `reno.plot_refs_single_axis(run1, [predator_prey.foxes, predator_prey.rabbits])`*, a utility function that helps visualize multiple model components on one set of axes. This is a common plot for system dynamics models because the relationship between two components may be more important than the relative scale differences. Note that each oscillation appears to have wider fluctuation: Reno simulates discrete timesteps, which means for a large* `dt` *timestep, errors can accumulate in some types of feedback loops.*

In large systems, it can be difficult to follow certain relationships from a collection of plots. Stock and flow diagrams can be configured to include inline sparklines of relevant stocks and flows to produce an output as shown in Figure 8.

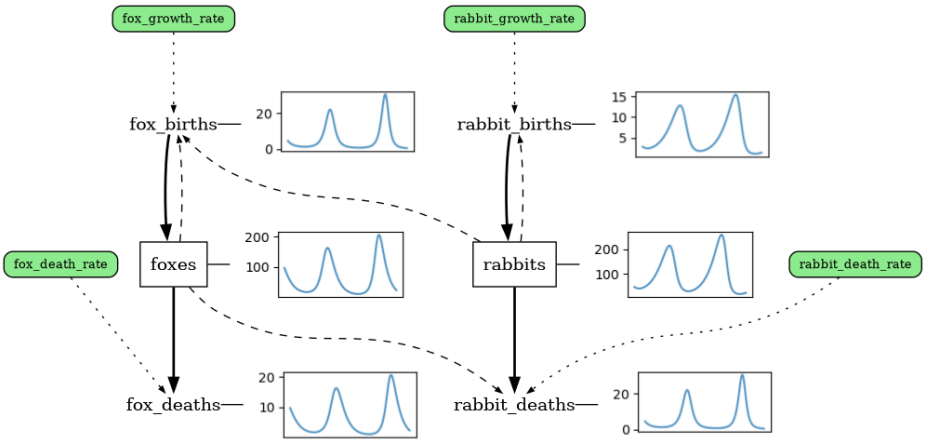
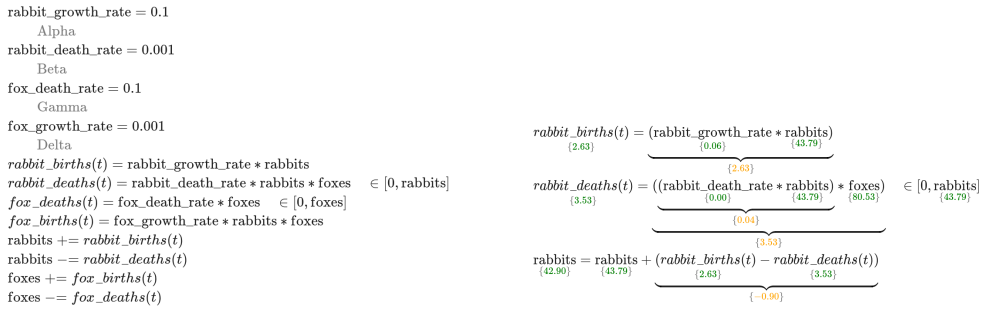


Figure 8. *Graphviz output from* `predator_prey.graph(stock_sparklines=True, flow_sparklines=True)`

Reno can output a latex representation of all of the equations involved in the system, which can be viewed natively within Jupyter or exported as a string for a latex paper. Additionally, an individual sample and timestep can be passed as parameters, which will then output the equations with the computed value from each section of each equation explicitly written out. This can be useful for diagnosing unexpected or incorrect equation results.



(a) (b)

Figure 9. *Latex outputs for a model. These render interactively in Jupyter [22] notebooks and can be exported as raw strings for inclusion in latex documents.*

3.4. Bayesian inference

To run Bayesian inference on a model, at least one model parameter must be defined with a prior probability distribution. The model can be converted and simulated within a PyMC model by calling the `.pymc()` function, which has a signature that mirrors the base simulation call. This means that it can take parameter configurations and settings to control the MCMC sampling process. The `.pymc()` function also takes observed data values and uncertainties by supplying `Observation` objects to the `observations` parameter, shown in Program 4. Reno turns these observations into gaussian likelihood functions, with mean values centered around the observed equation (connecting it to the system simulation), a user-specified standard deviation to allow for uncertainty in the observations (`1.0` in the example in Program 4), and the actual observed values for PyMC to target (`[100]` in the example). These likelihood functions are used by PyMC during its MCMC sampling process to approximate the posteriors. Passing an `n` to the `.pymc()` call configures the total number of samples produced by the sampler. By default, four chains are used, but this and all other PyMC sampler settings (such as the random seed) can be configured by passing a `sampling_kwargs` dictionary. The sampler used by default is the Sequential Monte Carlo[7], but the standard samplers can be used as well by passing `smc=False`.

```
run3 = predator_prey.pymc(
    n=2000,
    rabbit_growth_rate=.05,
    rabbits_0=75.0,
    observations=[r.Observation(predator_prey.foxes.maximum(), 1.0, [100])],
)
```

Program 4. *An example PyMC-enabled run, targeting an observed maximum fox population value of `100`, with a standard deviation (uncertainty allowance) of `1.0`.*

The output from a `.pymc` run is an Arviz `InferenceData`, a specialized XArray dataset that includes both a `.prior` section as well as `.posterior`.

3.5. A complete example run with Bayes

This section uses an example from the [23] textbook, specifically a “one-compartment model” or a model of the concentration of the drug Dilantin in the system under repeated dosages at a given interval. Parameters of this model include the elimination half-life of the medication in the blood plasma, the absorption fraction or how much of the ingested medication reaches the blood stream, and dosage amount and interval. In this hypothetical example, we use one or more noisy measurements of the concentration to demonstrate how variables we treat as uncertain (artificially in this case) can be calibrated or refined.

[7] https://www.pymc.io/projects/examples/en/latest/samplers/SMC2_gaussians.html

Program 5 shows the Reno model definition for the one-compartment model with variables configured for the "ground truth" default run.

```
import reno as r

compartment = r.Model("compartment", steps=168)
with compartment:
    # define the three main components, the amount of drug in the system
    # and the flows that increase/decrease it
    drug_in_system = r.Stock(doc="Mass of medication in blood serum.")
    ingested = r.Flow(doc="Pulsed inflow of medication when dosage is taken.")
    eliminated = r.Flow(doc="Rate of change of drug leaving system.")

    # hook up the flows to the stock
    ingested >> drug_in_system >> eliminated

    # define relevant variables
    absorption_fraction = r.Variable(.12)
    dosage = r.Variable(100 * 1000, doc="Dosage is 100 * 1000 micrograms")
    start = r.Variable(0, doc="Timestep of first dosage. (in hours)")
    interval = r.Variable(8, doc="Timesteps between each dosage. (in hours)")

    volume = r.Variable(3000, doc="Volume of blood serum, 3000 mL")
    concentration = r.Variable(drug_in_system / volume)
    half_life = r.Variable(22, doc="Half-life of medication. (in hours)")
    elimination_constant = r.Variable(-r.log(0.5) / half_life)

    # set the equations for the flows based on above variables
    eliminated.eq = elimination_constant * drug_in_system
    ingested.eq = absorption_fraction * dosage * r.repeated_pulse(start, interval)
```

Program 5. *Reno implementation of a one-compartment model for Dilantin.*

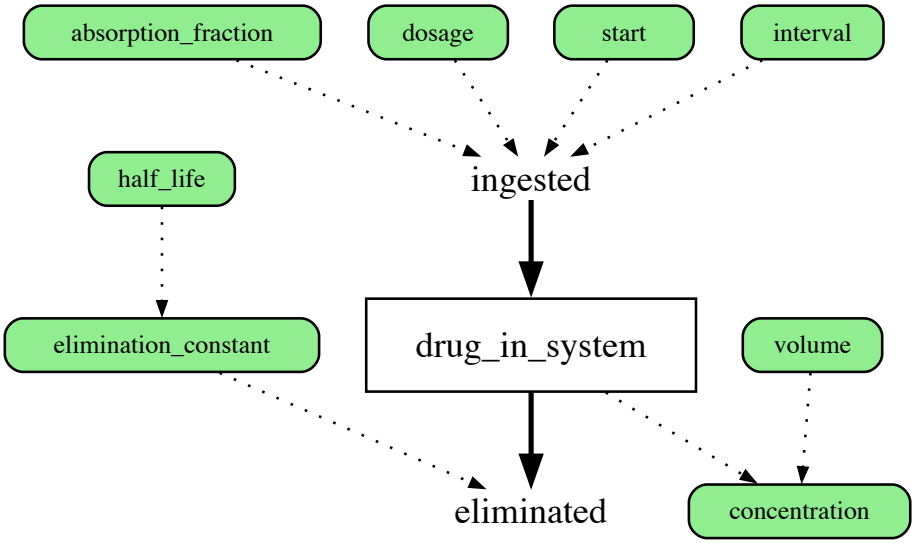


Figure 10. *The stock and flow diagram for the one-compartment model of Dilantin.*

We assume there are three uncertain variables: the first dosage time, the exact dosage interval, and the absorption fraction. We additionally assume we have collected measurements of the concentration at three different times. We retrieve "ground truth" data by running the model as is and obtaining those three values, shown offset by a small amount of noise as the black dots in Figure 11. Ground truth concentration measurements at timesteps 30, 100, and 150 are 9.66949, 15.5064, and 14.9687, respectively. In this example, we assume a lower resolution measurement to highlight the allowance for uncertainty in the data by using 9.5, 15.5, and 15.0.

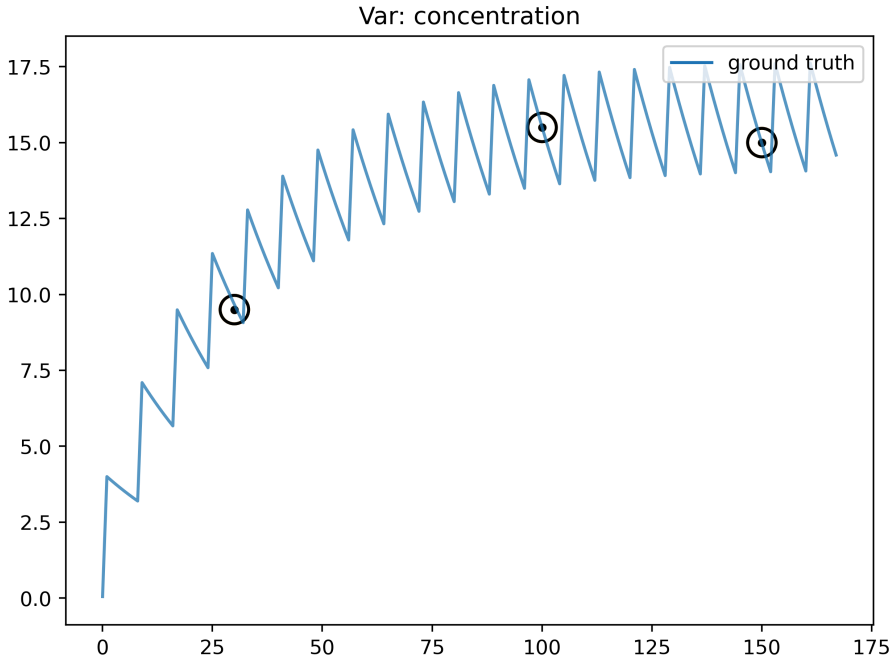


Figure 11. *Concentration timeseries values when running* `compartment()`*. The plot was created with* `reno.plot_trace_refs(compartment, [compartment()], [compartment.concentration])`*. The black points are added to show the measured data that will be used.*

When starting with an uncalibrated model, or if there is uncertainty about some parameters, we set prior distributions for those parameters. Here, we use discrete uniform distributions for the start time and interval and a normal distribution for the absorption fraction. If we were to simply simulate forward passes through the model, randomly drawing from these distributions would produce the results shown in Figure 12.

```
compartment.pymc(
  n=4000,
  start=r.DiscreteUniform(0, 1),
  interval=r.DiscreteUniform(7, 9),
  absorption_fraction=r.Normal(.15, .025),
  compute_prior_only=True,
    sampling_kwargs=dict(random_seed=13)
)
```

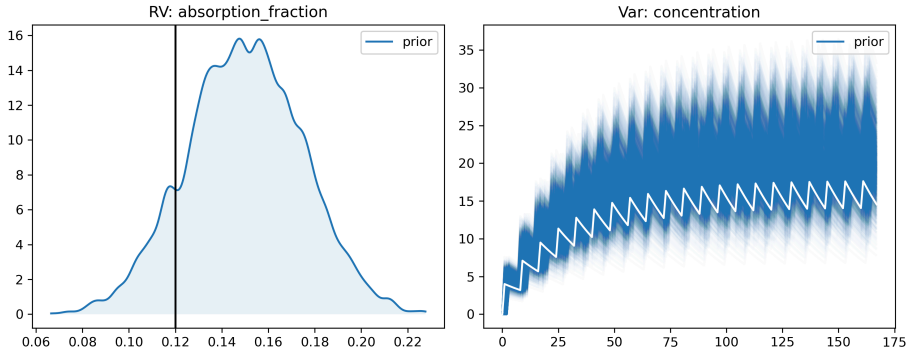


Figure 12. *Sampled distribution of the absorption fraction input parameter and simulated timeseries of the concentration variable. The original ground truth simulation run is highlighted in white and the target absorption fraction value is highlighted in black.*

When adding an observation for the concentration at one point in time, we can run the PyMC simulation and retrieve an approximated posterior distribution for the absorption fraction, highlighted in Figure 13. The probability mass for the single observation run starts converging to two different possible values: the correct one, and a different fraction for one of the other possible dosage intervals that would also result in the measured concentration value.

```
compartment.pymc(
  n=4000,
  start=r.DiscreteUniform(0, 1),
  interval=r.DiscreteUniform(7, 9),
  absorption_fraction=r.Normal(.15, .025),
  observations=[
    r.Observation(compartment.concentration.timeseries[100], 1, [15.5]),
  ],
    sampling_kwargs=dict(random_seed=13)
)
```

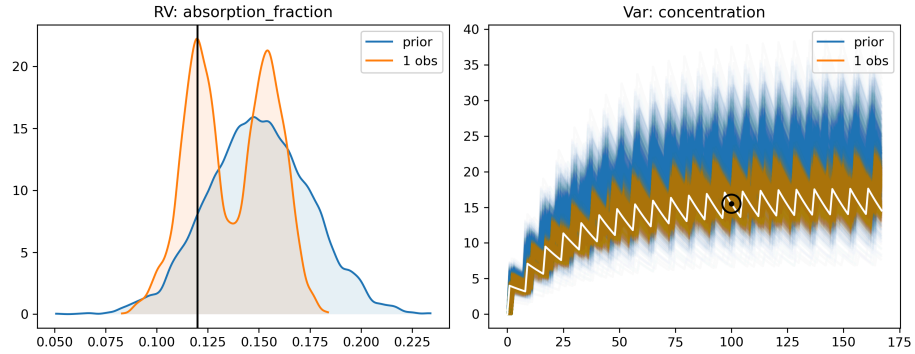


Figure 13. *Posterior distribution of the absorption fraction input parameter given one concentration timeseries observation (pinpointed in black). Ground truth is highlighted in black for absorption fraction and white for concentration.*

Supplying more data (the other two observed measurements) and rerunning the simulation results in what is shown in Figure 14, where the posterior probability distribution for the absorption_fraction is tightly concentrated around the ground truth value of 0.12.

```
compartment.pymc(
  n=4000,
  start=r.DiscreteUniform(0, 1),
  interval=r.DiscreteUniform(7, 9),
  absorption_fraction=r.Normal(.15, .025),
  observations=[
    r.Observation(compartment.concentration.timeseries[100], 1, [15.5]),
    r.Observation(compartment.concentration.timeseries[150], 1, [15.0]),
    r.Observation(compartment.concentration.timeseries[30], 1, [9.5]),
  ],
    sampling_kwargs=dict(random_seed=13)
)
```

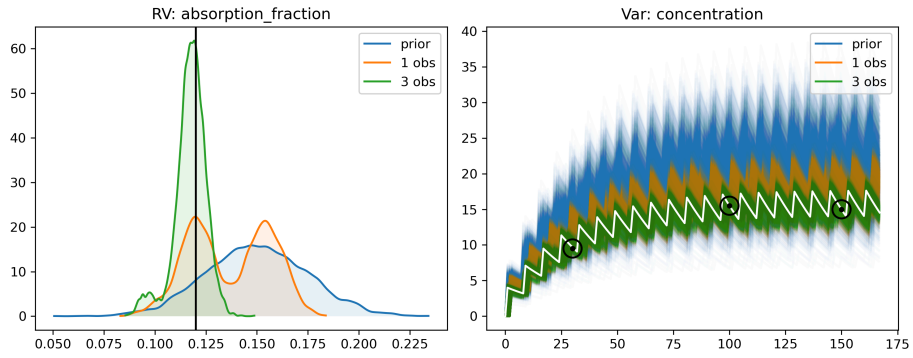


Figure 14. *Posterior distribution of the absorption fraction input parameter given three concentration timeseries observations (pinpointed in black). Ground truth is highlighted in black for absorption fraction and white for concentration.*

4. Technical implementation of PyMC integration

Due to the structure of the equation trees, converting individual equations into their PyMC equivalents is straightforward. Every `EquationPart` (the tree data structure class) has a `pt()` function which recursively calls throughout the entire tree, thus creating a PyTensor equation that mirrors the original. Much of the value that Reno provides is in the surrounding setup for the entire model, which can be complicated to achieve in PyMC because of the amount of boilerplate. A full model is simulated by running the various equations at each timestep in sequence, so translating this into PyTensor requires setting up the difference equations in a separate function and using their `scan`[8] operation.

One example of a challenging ability to support in raw PyMC is dynamically indexing into historical timeseries values inside of a `scan` function. By default, this target function only has access to the most recent ($t-1$) timestep for each value. PyMC supports passing in a specified set of previous timesteps, referred to as "taps", but the possibility of arbitrarily indexing them based on other variables in the system requires passing every previous timestep and separate equations to correctly convert the requested index, increasing the complexity of the code. Reno thus handles this infrastructure necessary to convert the model into PyMC, and provides an abstraction where only the components and the equations themselves need to be defined.

The overall flow of the conversion process is roughly implemented as follows:

1. Define the initial PyMC variables for each component of the Reno model and add the equations for their initial values. These definitions are appropriately ordered to account for any dependencies between the variables/initial equations.

[8] https://pytensor.readthedocs.io/en/latest/library/scan.html

2. All sequences are formatted as necessary and passed into PyMC's scan function, with the target step function defined with the following:
    1. All component values from the previous step, and any former steps if operations that look back in time are used, are pulled out of the parameters.
    2. Each component's equation is run (the equations are dependency ordered), stock values are updated, and the results of all of these are returned for handling by PyMC's scan function architecture.
3. All full timeseries sequences of every component are collected, and any final metric equations are run.

The conversion process results in python code with the following rough format:

```
def step_function(*args):
  # *args is populated by pytensor's `scan`, containing
  # previous timestep values, historical tap values where
  # necessary, and any additional sequence or static values
  s = args[0]
  ...

  # difference/update equations for each stock/flow/variable
  s_next = s + ...
  ...

  # return all updated values (pymc/pytensor has specific boilerplate to use)
    return [s_next, ...], ...

with pymc.Model() as model:
  # define all pymc distributions/variables and initial stock/flow values
  # e.g. for stock "s":
  s_init = pm.Deterministic("s_init", ...)
  ...

  # run the scan function to generate full sequence data for simulation
  [s_seq, ...] = pytensor.scan(fn=step_function, [s_init, ...], ...)

  # create pymc containers for the full sequence data (this way they get
  # added into any output arviz datasets from pymc sampling operations)
  s = pm.Deterministic("s", pt.concatenate([s_init, s_seq], ...)

  # any metric equations are added at the end
  final_metric = pm.Deterministic("final_metric", ...)
```

Program 6. *The roughly equivalent code that would construct a PyMC model in the same way as is returned from* `reno.pymc.to_pymc_model`*, or the string of raw code returned from* `reno.pymc.to_pymc_model_str`.

Reno can either directly set up a PyMC model using the above process, or output a raw string of Python code that would construct the equivalent PyMC model. This is valuable for situations where more complex PyMC-specific code is warranted and easier debugging.

5. Conclusion

Reno is an open-source Python library introduced to support building system dynamics models and applying Bayesian inference to the simulation process. The Python API is intended to support integration with additional software tooling, such as front-end interactive visualization capabilities. Future work will include the ability to directly integrate model context protocol (MCP) tools for language models, tooling to run within HPC environments, and improved web-based front-end tools.

Acknowledgements

This manuscript has been authored in part by UT-Battelle, LLC, under contract DE-AC05-00OR22725 with the US Department of Energy (DOE). The US government retains and

the publisher, by accepting the work for publication, acknowledges that the US government retains a non-exclusive, paid-up, irrevocable, world-wide license to publish or reproduce the submitted manuscript version of this work, or allow others to do so, for US government purposes. DOE will provide public access to these results of federally sponsored research in accordance with the DOE Public Access Plan (http://energy.gov/downloads/doe-public-access-plan).

This work was supported by the NNSA Office of Defense Nuclear Nonproliferation Research & Development.

References

[1] B. J. Alder and T. E. Wainwright, "Studies in Molecular Dynamics. I. General Method," *The Journal of Chemical Physics*, vol. 31, no. 2, pp. 459–466, 1959, doi: 10.1063/1.1730376.

[2] P. Hogeweg, "The roots of bioinformatics in theoretical biology.," *PLoS computational biology*, vol. 7, no. 3, p. e1002021, 2011, doi: 10.1371/journal.pcbi.1002021.

[3] K. Judd, *Numerical Methods in Economics*. in Mit Press. MIT Press, 1998. [Online]. Available: https://books.google.com/books?id=9Wxk_z9HskAC

[4] M. W. Macy and R. Willer, "From Factors to Actors: Computational Sociology and Agent-Based Modeling," *Annual Review of Sociology*, vol. 28, no. Volume28, 2002, pp. 143–166, 2002, doi: https://doi.org/10.1146/annurev.soc.28.110601.141117.

[5] Abril-Pla O, "PyMC: a modern, and comprehensive probabilistic programming framework in Python," *PeerJ Computer Science*, 2023, doi: 10.7717/peerj-cs.1516.

[6] M. J. Radzicki, *System Dynamics and Its Contribution to Economics and Economic Modeling*. New York, NY: Springer US, 2020, pp. 401–415. doi: 10.1007/978-1-4939-8790-0_539.

[7] R. Martin and M. Schlüter, "combining system dynamics and agent-based modeling to analyze social-ecological interactions—an example from modeling restoration of a shallow lake," *Frontiers in Environmental Science*, 2015, doi: 10.3389/fenvs.2015.00066.

[8] D. C. Lane, "The power of the bond between cause and effect: Jay Wright Forrester and the field of system dynamics," *System Dynamics Review*, vol. 23, no. 2–3, pp. 95–118, 2007, doi: https://doi.org/10.1002/sdr.370.

[9] H. Akkermans and N. Dellaert, "The rediscovery of industrial dynamics: the contribution of system dynamics to supply chain management in a dynamic and fragmented world," *System Dynamics Review*, vol. 21, no. 3, pp. 173–186, 2005, doi: https://doi.org/10.1002/sdr.317.

[10] M. Danesh Shakib, "Using system dynamics to evaluate policies for industrial clusters development," *Computers & Industrial Engineering*, vol. 147, p. 106637, 2020, doi: https://doi.org/10.1016/j.cie.2020.106637.

[11] M. Özbayrak, T. C. Papadopoulou, and M. Akgun, "Systems dynamics modelling of a manufacturing supply chain system," *Simulation Modelling Practice and Theory*, vol. 15, no. 10, pp. 1338–1355, 2007, doi: https://doi.org/10.1016/j.simpat.2007.09.007.

[12] H. Husniah, N. Anggriani, and A. Supriatna, "System dynamics approach in managing complex biological resources," *ARPN Journal of Engineering and Applied Sciences*, vol. 10, pp. 1685–1690, 2015.

[13] A. H. P. Tan, E. H. Yap, and Y. A. Abakr, "A Complex Systems Analysis of the Water-Energy Nexus in Malaysia," *Systems*, vol. 8, no. 2, 2020, doi: 10.3390/systems8020019.

[14] I. Fornacon-Wood, H. Mistry, C. Johnson-Hart, C. Faivre-Finn, J. P. O'Connor, and G. J. Price, "Understanding the Differences Between Bayesian and Frequentist Statistics," *International Journal of Radiation Oncology*Biology*Physics*, vol. 112, no. 5, pp. 1076–1082, 2022, doi: https://doi.org/10.1016/j.ijrobp.2021.12.011.

[15] C. Robert and G. Casella, "A Short History of Markov Chain Monte Carlo: Subjective Recollections from Incomplete Data," *Statistical Science*, vol. 26, no. 1, pp. 102–115, 2011, doi: 10.1214/10-STS351.

[16] A. E. Gelfand and A. F. M. Smith, "Sampling-Based Approaches to Calculating Marginal Densities," *Journal of the American Statistical Association*, vol. 85, no. 410, pp. 398–409, 1990, doi: https://doi.org/10.2307/2289776.

[17] S. R. Penk *et al.*, "A system dynamics model to understand the integrated ecological and human dimension aspects of wildlife health and disease management," *bioRxiv*, 2025, doi: 10.1101/2025.09.22.677405.

[18] J. Andrade and J. Duggan, "A Bayesian approach to calibrate system dynamics models using Hamiltonian Monte Carlo," *System Dynamics Review*, vol. 37, no. 4, pp. 283–309, 2021, doi: https://doi.org/10.1002/sdr.1693.

[19] S. Fortmann-Roe, "Insight Maker: A general-purpose tool for web-based modeling & simulation," *Simulation Modelling Practice and Theory*, vol. 47, pp. 28–45, 2014, doi: https://doi.org/10.1016/j.simpat.2014.03.013.

[20] E. Martin-Martinez, R. Samsó, J. Houghton, and J. Solé, "PySD: System Dynamics Modeling in Python," *Journal of Open Source Software*, vol. 7, no. 78, p. 4329, 2022, doi: 10.21105/joss.04329.

[21] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020, doi: https://doi.org/10.1038/s41586-020-2649-2.

[22] T. Kluyver *et al.*, "Jupyter Notebooks - a publishing format for reproducible computational workflows," in *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, F. Loizides and B. Scmidt, Eds., Netherlands, 2016, pp. 87–90. [Online]. Available: https://eprints.soton.ac.uk/403913/

[23] A. B. Shiflet and G. W. Shiflet, *Introduction to computational science: modeling and simulation for the sciences*. Princeton University Press, 2014, pp. 48–50. [Online]. Available: https://assets.press.princeton.edu/chapters/s2_10291.pdf